MindForge LABS

DAY 1

Python Introduction and Language Basics

An interactive study guide: click to navigate, predict outputs, answer quick checks, and write your own practice solutions right in the PDF.

For students new to Python | Works in Adobe Acrobat Reader, Chrome, Edge, Firefox

Contents

Click any section to jump to it. Use **Contents** in the page footer to come back here, or open the bookmarks panel in your PDF viewer.

1	What is Python?
2	Your first Python code
3	Comments, statements, and indentation
4	Variables and assignment
5	Built-in data types
6	Keywords and identifiers
7	Operators
8	Type conversion
9	Input and formatted output
10	Practice tasks and recap
	Answer Key

Learning outcomes

Tick each box when you can do it confidently:

- [ ] Explain what Python is and name common use cases.
- [ ] Run a notebook cell and a .py script in VS Code.
- [ ] Use print(), comments, variables, and built-in types.
- [ ] Recognize keywords and valid identifiers.
- [ ] Perform basic operations, accept input, and format output.

> ✎ **HOW THIS GUIDE WORKS**
> **Yellow boxes** are multiple-choice quick checks. **Teal boxes** are fill-in fields. Answers live in the Answer Key and every answer links back to where you were. *Tip: predict the output before you read it.*

Section 1

What is Python?

Python is a high-level, general-purpose programming language with readable syntax.

- **High-level:** code uses concepts such as lists and dictionaries rather than requiring manual memory management.
- **Interpreted workflow:** the Python runtime executes code and reports errors.
- **Dynamically typed:** names can refer to objects of different types at different times.
- **Multi-paradigm:** supports procedural, object-oriented, and functional programming.

Why Python?

- Readable syntax helps beginners focus on problem-solving.
- Large standard library and third-party ecosystem.
- Useful for automation, web backends, data analysis, AI/ML, testing, and scripting.
- Trade-off: pure Python may be slower than compiled languages for CPU-intensive tasks.

Where is Python used?

Area	Example
Automation	Rename files or generate reports
Web development	Build APIs and server-side applications
Data analysis	Clean and summarize datasets
AI / machine learning	Experiment with models
Testing	Automate checks for software
Education	Teach programming concepts

✎ **QUICK CHECK** **Question 1**

Python being "dynamically typed" means:

- You must declare every variable's type before using it
- A name can refer to objects of different types at different times
- It can only store numbers
- It is compiled before it runs

Reveal answer & explanation (opens the Answer Key)

Section 2

Your first Python code

In VS Code, install Python 3 and the Python extension, then select your interpreter. To run a script, save code in a `.py` file and use **Run Python File in Terminal**. In a notebook, execute a cell with **Shift + Enter**.

```
print("Hello, Python!")
print("I am learning programming.")
print(2 + 3)
```

```
OUTPUT
Hello, Python!
I am learning programming.
5
```

✎ **PREDICT FIRST** **What is different between printing text and calculating 2 + 3?**

✎ **QUICK CHECK** **Question 2**

What does print(2 + 3) display?

- 2 + 3
- 23
- 5
- Nothing; it raises an error

Reveal answer & explanation (opens the Answer Key)

Section 3

Comments, statements, and indentation

- A comment begins with #; it explains code and is not executed.
- Python uses indentation to group statements into blocks.
- Use four spaces per indentation level.
- A colon (`:`) introduces a block after statements such as `if`, `for`, `def`, and `class`.

```
# This is a comment
student_name = "Asha"

if student_name:
    print(f"Hello, {student_name}!")
```

```
OUTPUT
Hello, Asha!
```

✎ **QUICK CHECK** **Question 3**

Which symbol introduces an indented block after if, for, def or class?

- Semicolon ;
- Colon :
- Hash #
- Brace {

Reveal answer & explanation (opens the Answer Key)

MindForge LABS

Section 4

Variables and assignment

A variable is a **name that refers to an object**. The = operator assigns a value to a name; it does not test equality. Good names describe the purpose of the value. Python names are case-sensitive: `score` and `Score` are different.

```
student_name = "Asha"
age = 19
temperature = 28.5
is_enrolled = True

print(student_name)
print(age)
print(temperature)
print(is_enrolled)
```

```
OUTPUT
Asha
19
28.5
True
```

✎ **TRY IT** **Create variables for your name, favorite subject, and hours you study each week. Print them in a single sentence. Write your code here:**

✎ **QUICK CHECK** **Question 4**

In Python, what does the = operator do?

- Tests whether two values are equal
- Assigns a value to a name
- Adds two numbers
- Compares types

Reveal answer & explanation (opens the Answer Key)

Section 5

Built-in data types

Type	Example	Typical use
int	42	Whole numbers
float	3.14	Decimal numbers
str	"Python"	Text
bool	True	Logical conditions
NoneType	None	No value / absence of a value

```
price = 49.50
language = "Python"
is_ready = False
not_set = None

print(type(price))
print(type(language))
print(type(is_ready))
print(type(not_set))
```

```
OUTPUT
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

✎ **QUICK CHECK** **Question 5**

What is type(49.50)?

- ○
- ○
- ○
- ○

Reveal answer & explanation (opens the Answer Key)

Section 6

Keywords and identifiers

Identifier: a name chosen for a variable, function, class, or other object. Rules:

- May contain letters, digits, and underscores.
- Cannot begin with a digit.
- Cannot contain spaces or hyphens.
- Is case-sensitive.
- Cannot be a reserved keyword.

Keyword: a reserved word with special meaning, such as `if`, `for`, `class`, `return`, `True`, and `None`. Soft keywords such as `match` and `case` are used only in specific syntax contexts.

Valid	Invalid
total_marks, _count, student2	2score, student-name, class

```
# Valid identifiers
total_marks = 95
student2 = "Ravi"
_count = 3

print(total_marks, student2, _count)

# Uncomment the next line to see a SyntaxError:
# class = "Python"
```

```
OUTPUT
95 Ravi 3
```

✎ **QUICK CHECK** **Question 6**

Which of these is a valid identifier?

- 2score
- student-name
- class
- student2

Reveal answer & explanation (opens the Answer Key)

MindForge LABS

Section 7

Operators

- **Arithmetic:** `+ - * / // % **`
- **Comparison:** `== != < <= > >=`
- **Logical:** `and or not`

Remember: = assigns a value, while == compares values.

```
a, b = 7, 2

print("Addition:", a + b)
print("Division:", a / b)
print("Floor division:", a // b)
print("Remainder:", a % b)
print("Power:", a ** b)
print("Is a greater than b?", a > b)
print("Are they equal?", a == b)
```

```
OUTPUT
Addition: 9
Division: 3.5
Floor division: 3
Remainder: 1
Power: 49
Is a greater than b? True
Are they equal? False
```

✎ **PREDICT FIRST** **Predict the output of: print(7 / 2, 7 // 2, 7 % 2)**

✎ **QUICK CHECK** **Question 7**

What is 7 // 2 in Python?

- 3.5
- 3
- 4
- 1

Reveal answer & explanation (opens the Answer Key)

Section 8

Type conversion

Convert values when an operation requires a different type. Some conversions can fail, so validate user input in real programs.

```
age_text = "19"
age = int(age_text)
height = float("1.72")
message = "Age: " + str(age)

print(age + 1)
print(height)
print(message)

# Try int("hello") to observe a ValueError (run separately).
```

```
OUTPUT
20
1.72
Age: 19
```

✎ **QUICK CHECK** **Question 8**

What does int("19") + 1 evaluate to?

- "191"
- 20
- A TypeError
- 19.1

Reveal answer & explanation (opens the Answer Key)

MindForge LABS

Section 9

Input and formatted output

`input()` always returns a string. Convert it before doing numeric calculations. An f-string inserts expressions into text.

```
name = input("Your name: ").strip()
age_text = input("Your age: ").strip()

try:
    age = int(age_text)
except ValueError:
    print("Please enter your age as a whole number.")
else:
    print(f"Hello, {name or 'student'}!")
    print(f"Next year you will be {age + 1}.")
```

```
OUTPUT
Please enter your age as a whole number.
(shown when the age typed is not a whole number)
```

✎ **QUICK CHECK** **Question 9**

What type of value does input() always return?

- int
- float
- str
- bool

Reveal answer & explanation (opens the Answer Key)

Section 10

Practice tasks

Work in VS Code, then record your answers here.

✎ **TRY IT** **Task 1. Create variables for a book's title, price, and availability.**

✎ **TRY IT** **Task 2. Ask the user for two numbers and print their sum.**

✎ **TRY IT** **Task 3. Convert Celsius to Fahrenheit using F = C * 9 / 5 + 32.**

✎ **TASK 4** **Tick the VALID identifiers**

- [ ] `first_name`
- [ ] `first-name`
- [ ] `2nd_place`
- [ ] `class`
- [ ] `score2`

Check answer

✎ **PREDICT FIRST** **Task 5. Predict the output before running: print(7 / 2, 7 // 2, 7 % 2)**

Quick recap

- Python is used for scripting, automation, web development, data, and more.
- Variables are names bound to values/objects.
- Types determine what operations make sense.
- Keywords are reserved; identifiers are names you create.
- Input is text until converted.

✎ **EXPLAIN IT** **Instructor prompt**
Explain one answer aloud, then modify a code example to test your understanding.

Answer Key

Click the section link to return to where you were.

Section 1: Python being "dynamically typed" means:

Answer: B. A name can refer to objects of different types at different times
Dynamic typing: the type belongs to the object, so the same name can later refer to an object of another type. Back to section 1

Section 2: What does print(2 + 3) display?

Answer: C. 5
Python evaluates the expression 2 + 3 first, then prints the result: 5. Text in quotes would print as-is. Back to section 2

Section 3: Which symbol introduces an indented block after if, for, def or class?

Answer: B. Colon :
A colon starts the block; the indented lines (four spaces) that follow belong to it. # starts a comment. Back to section 3

Section 4: In Python, what does the = operator do?

Answer: B. Assigns a value to a name
= assigns. To test equality, use ==. Back to section 4

Section 5: What is type(49.50)?

Answer: C.
A number with a decimal point is a float. Back to section 5

Section 6: Which of these is a valid identifier?

Answer: D. student2
Identifiers cannot start with a digit, contain hyphens, or be reserved keywords like class. Back to section 6

Section 7: What is 7 // 2 in Python?

Answer: B. 3
// is floor division: 7 / 2 = 3.5, rounded down gives 3. (% gives the remainder, 1.) Back to section 7

Section 8: What does int("19") + 1 evaluate to?

Answer: B. 20
int("19") converts the text to the number 19, so adding 1 gives 20. Back to section 8

Section 9: What type of value does input() always return?

Answer: C. str
input() returns a string. Convert it (e.g. int(...)) before doing numeric calculations. Back to section 9

Section 10: Task 4 and Task 5

Task 4 valid identifiers: first_name and score2. first-name has a hyphen, 2nd_place starts with a digit, and class is a keyword.
Task 5: prints `3.5 3 1`. Back to practice